

Testing

Ensures that software works as expected by validating features against requirements. It helps catch bugs early, improves reliability, and maintains high-quality standards in development.

How to do testing?

1. Clone the repository, then build and run the submitted code.
2. Agree on your teamwork: how do you divide testing between reviewers?
3. Test functionality and check compliance with the requirements.
4. Provide feedback in the group chat and request fixes if necessary.
5. Clearly state what changes are mandatory and what are optional fixes.
6. Repeat the testing cycle after submitters make the requested changes for as many times as is needed.

Mandatory

All required deliverables are present in the repository

Confirm each deliverable exists before reviewing further: the working CLI application, a committed test coverage report, a seeded database file, and an ER diagram of all three tables.

Relaunching with an existing database loads the saved account and goes straight to the main menu without prompting again.

Relaunch the app and verify no username prompt appears, the saved account loads straight to the main menu.

Listing an item stores it and returns to the main menu, and it appears in 'view my inventory' with status 'available', its detail view showing the current user as owner.

From the main menu select 'list an item', enter name, description, and cost per day, and confirm the main menu reappears. Open 'view my inventory' and verify the item shows with status 'available', then open its detail and verify the owner is the current user's username.

Selecting 'exit' shuts the program down cleanly.

Pick 'exit'. The program should stop right away and return you to your normal terminal prompt, with no freezing or hanging.

Every list view (view my inventory, record a rental, confirm a return) shows one row per item, offers a back-to-menu option, and paginates with next/previous past 5 items.

The displayed fields differ by list: name and status in inventory, name and cost per day in record a rental, item name and renter in confirm a return. Use 'view my inventory' (with at least 6 items listed) to confirm next/previous paging past 5.

Selecting an item from the inventory opens a detail view showing name, description, cost per day, status, owner, and listing date.

Delisting an item from its detail view sets its status to 'unlisted' and removes it from the 'record a rental' list.

Delist an item. Verify it shows as 'unlisted' in 'view my inventory' and does not appear in 'record a rental'.

Relisting a delisted item returns its status to 'available', and relisting is blocked while the item still has an active rental.

Relist a delisted item with no active rental and verify it returns to 'available' and reappears in 'record a rental'. Then delist an item that is currently rented and verify relisting it is refused until the rental is returned.

Recording a rental with a new renter sets the item's status to 'rented', removes it from the rental list, and creates a renter record.

Record a rental using a renter name not yet in the system. Verify the item shows 'rented' in inventory, is absent from 'record a rental', and a new renter row exists.

Recording a rental with an existing renter name reuses that record instead of creating a duplicate.

Record two rentals using the same renter name. Run `sqlite3 <your-db>.db "SELECT COUNT(*) FROM users WHERE username = '<renter name>';"` and verify it returns 1 (the renter wasn't duplicated). Then record a rental with a different new name and run the same query for that name to confirm it returns 1, a separate renter row.

The 'record a rental' list shows only items with status 'available'.

Set up at least one rented item and one unlisted item, then open 'record a rental'. Verify neither appears.

Confirming a return sets the item's status back to 'available', writes the return timestamp, and keeps the rental record.

Confirm a return. Verify the item reappears in 'record a rental'. Run `sqlite3 <your-db>.db "SELECT rental_id, returned_at, status FROM rentals;"` and verify

returned_at is set, status is 'closed', and the record still exists.

The 'confirm a return' list is ordered by due date, and a rental's detail view shows the item name, renter's username, start and end times, and status.

With at least two active rentals due on different dates, open 'confirm a return' and verify they're listed earliest-due first. Open a rental and verify its detail view shows the item name, the renter's username, the start and end times, and the status.

Returning an item that was delisted while rented leaves its status 'unlisted' rather than 'available'.

Rent an item, delist it while the rental is active, then confirm the return. Verify the item's status stays 'unlisted'.

Running the test suite passes, and each test sets up and tears down its own state so tests run independently.

Run the full suite and verify it passes. Run a single test on its own and verify it still passes without depending on other tests having run first. Confirm tests run against a real SQLite instance (a temporary file wiped after each run), not a mock, and that each test starts from fresh state.

A test coverage report is committed and shows 100% coverage.

Open the committed coverage report, then run the coverage tool live and confirm it matches, so you know it's up to date. It should report 100% coverage.

Data-layer tests cover insert and query (by ID and by filter) for users, items, and rentals, status updates for items and rentals, and incorrect inputs that fail correctly without crashing.

Inspect the suite and verify insert and query (by ID and by filter) are tested for all three tables, that status updates are tested for items and rentals (the users table has no status column), and that incorrect inputs — such as a lookup by an ID that doesn't exist — are tested and fail correctly without crashing (the specific typed error, not a crash or wrong result).

Constraint tests fail as expected for a duplicate username (UNIQUE), a rental referencing a non-existent item (foreign key), an item missing a required field (NOT NULL), and an out-of-range status (CHECK).

Verify a test exists for each of the four cases, and that each asserts on the specific error for the rule it broke (the typed error), not merely that the write failed.

Database failures are caught in the repository and returned as typed errors, not left to propagate raw.

Ask the author(s) to walk you through what happens when a query fails — a missing row, a broken constraint, a bad connection. Then ask what the service layer actually receives back. They can name the specific typed error for at least one failure kind and show it's caught in the repository, not surfaced as a raw driver/SQL error.

Behaviour tests cover account create-then-load, the item status transitions, and the rental business rules.

Verify tests exist for the account (created on first run, loaded without duplication on the next), the status transitions (list → available, rent → rented, return → available or unlisted), and the business rules (no re-renting, no renting a delisted item, existing renter reused, end time equals start time plus the days entered).

The state machine's allowed transitions are enforced deliberately in code, not left to the database.

Ask the author(s) to walk you through where the allowed transitions are enforced in the code. Then ask what would happen to an invalid transition if that check were removed. They can point to the service layer (not the database) and name a transition the check blocks.

The repository contains an ER diagram showing all three tables with columns, types, constraints, and relationships.

Each schema constraint was chosen on purpose, to prevent a specific bad state.

Ask the author(s) to walk you through 2-3 constraints spanning different types — e.g. a UNIQUE, a foreign key, and a CHECK. Then ask what bad state each one prevents. They can name the constraint type and state precisely what invalid state it blocks, not just that it exists.

The committed database file contains at least 6 listed items.

Run `sqlite3 <your-db>.db "SELECT COUNT(*) FROM listed_items;"` and verify the count is at least 6.

The layered architecture holds: each layer calls only the layer directly beneath it, and only the repository touches SQLite.

Trace the dependencies `Transport → Service → Repository → Infrastructure` and confirm no layer calls past the one beneath it (e.g. transport never calls the repository or

touches SQLite directly). Domain is the fifth layer, a set of shared types any layer may use, so referencing it is expected, not a skip. To trace, inspect each layer's imports and calls and confirm only the repository reaches the database.

The full lifecycle works end to end: first launch, listing an item, viewing inventory, recording a rental, confirming a return, and delisting.

Run through the full lifecycle end to end, in one continuous session: first launch through to the main menu, listing an item, viewing inventory to see it listed, recording a rental with a new renter, confirming the return, and delisting the item.

On a fresh database, the app prompts for a username and reaches the main menu after a name is entered.

Delete the database file and relaunch. Verify the app prompts for a username, and after entering one, proceeds to the main menu. Confirm the database file was created and contains the users, listed_items, and rentals tables (e.g. `sqlite3 <your-db>.db '.tables'`).

Extra

The author(s) can account for how they used AI while building this.

Ask the author(s) to walk you through where it was used and what they did with what it gave them, then ask about a time it produced something wrong or unhelpful and what happened next.

- 1 - little more than "we asked AI", or answers were taken as-is without checking.
- 2 - they cover what it was used for, but not how they judged the answers.
- 3 - they cover what they asked and why, how they checked the answer, and where they chose not to use it.

Git history shows work merged through reviewed pull requests, with small focused commits and clear messages.

- 1 - merged via PRs, but with no sign of review (self-merged, no comments or approval), and commits lump unrelated changes together with vague messages ("wip", "fix").
- 2 - PRs show some review before merging, with commits each scoped to one logical change, though messages aren't consistently clear.
- 3 - every PR is reviewed and approved before merging, commits are granular, and messages clearly say what changed and why.